Atelier Créactifs: Raspberry Pi

Séance 2: Le terminal

François Marelli
UMONS, ISIA Lab

9 février 2026

ATELIERS créactifs!

ISIA Lab
INFORMATION, SIGNAL, ARTIFICIAL INTELLIGENCE

UMONS
Université de Mons

Des problèmes courants

Quand on commence à utiliser un Raspberry Pi

Que faire quand:

- vous n’avez pas d’accès physique au Raspberry Pi
- l’outil graphique des paramètres système ne suffit pas
- des tâches simples sont répétitives et ennuyeuses
- les ressources doivent être économisées
- vous devez vous débrouiller avec un système Linux inconnu

Une solution commune: le terminal!

Plan de la séance

1. C’est quoi un terminal?
2. Naviguer dans le système de fichiers
3. Des outils pour se simplifier la vie
4. Contrôler le terminal à distance

C’est quoi un terminal?

Une interface de contrôle basée sur du texte

C'est moche, mais efficace

CLI (Command Line Interface) >< GUI (Graphical User Interface)

→ Pas d'interface graphique = pas de fenêtres, boutons ou souris!

Interface ancienne mais encore utilisée car elle:

- utilise peu de ressources
- donne accès à des outils système puissants
- permet d'automatiser des tâches
- permet de contrôler une machine à distance
- est disponible universellement indépendamment de l'interface graphique

Premiers pas

4 options pour ouvrir un terminal:

- cliquer sur l'icône du terminal dans la barre d'outils
- utiliser le menu principal
- utiliser un raccourci clavier: `Ctrl+Alt+T`
- changer de terminal virtuel avec `Ctrl+Alt+Fn` (bouée en cas de plantage GUI)

L'invite de commande (prompt): `rpi@rpi-01:~ $`

Premiers pas

4 options pour ouvrir un terminal:

- cliquer sur l'icône du terminal dans la barre d'outils
- utiliser le menu principal
- utiliser un raccourci clavier: `Ctrl+Alt+T`
- changer de terminal virtuel avec `Ctrl+Alt+Fn` (bouée en cas de plantage GUI)

L'invite de commande (prompt): `rpi@rpi-01:~ $`

- `rpi`: nom de l'utilisateur

Premiers pas

4 options pour ouvrir un terminal:

- cliquer sur l'icône du terminal dans la barre d'outils
- utiliser le menu principal
- utiliser un raccourci clavier: `Ctrl+Alt+T`
- changer de terminal virtuel avec `Ctrl+Alt+Fn` (bouée en cas de plantage GUI)

L'invite de commande (prompt): `rpi@rpi-01:~ $`

- `rpi`: nom de l'utilisateur
- `rpi-01`: nom de l'hôte

Premiers pas

4 options pour ouvrir un terminal:

- cliquer sur l'icône du terminal dans la barre d'outils
- utiliser le menu principal
- utiliser un raccourci clavier: `Ctrl+Alt+T`
- changer de terminal virtuel avec `Ctrl+Alt+Fn` (bouée en cas de plantage GUI)

L'invite de commande (prompt): `rpi@rpi-01:~ $`

- `rpi`: nom de l'utilisateur
- `rpi-01`: nom de l'hôte
- `~`: répertoire courant (~= home)

Anatomie d'une commande

Une commande = un programme + des options + des arguments

`ls` `-l Documents`

- `ls`: nom de la commande (list)
- `-l`: option
- `Documents`: argument ("entrée" principale de la commande)

`ls` sans arguments pour une liste simple du répertoire courant

Tip: utiliser `Tab` pour compléter les commandes et les chemins

La sortie de `ls`

Liste le contenu d'un répertoire

Par colonne:

1. type et permissions d'accès
2. nombre de liens vers cet élément
3. utilisateur propriétaire
4. groupe propriétaire
5. taille en octets
6. date de dernière modification
7. nom du fichier ou du répertoire

Intérpréter les permissions

Commande: ***`ls -l`***

1 lettre + 3 groupes de 3:

`d rwx r-x r-x`

1. `d`: type (d = dossier, - = fichier, l = lien)
2. `rwx`: permissions du propriétaire
3. `r-x`: permissions du groupe
4. `r-x`: permissions des autres utilisateurs

Permissions: r = lecture, w = écriture, x = exécution, - = pas de permission

Exécution pour un dossier = accès au contenu

Naviguer dans le système de fichiers

S’orienter dans les répertoires

Tous les chemins mèment à home

pwd (print working directory): afficher le *chemin* du répertoire courant

2 types de chemins:

- absolu: commence par `/` et explicite depuis la racine
- relatif: à partir du répertoire courant

Exemple:

- `/home/rpi/Documents` - absolu
- `Documents` - relatif (équivalent si on est dans `/home/rpi`)

Tip: contrairement à Windows, les chemins utilisent `/` et non `\`

Se déplacer dans les répertoires

`cd` *`chemin`* (change directory): se déplacer vers le chemin spécifié

Chemins relatifs locaux:

`.`	=	répertoire courant
`..`	=	répertoire parent

Exemple:

```
$ cd
$ cd ./Documents
$ cd ../Desktop
```

Tip: **`cd`** sans argument = retour au home

La racine

I am root

/ (root):

- est à l'origine de tous les chemins absolus
- est un répertoire comme les autres
- contient l'entièreté du système de fichiers

Que contient la racine?

```
$ cd /
$ ls -l
```

Tip: *root* est le nom du super-utilisateur (administrateur)

L'arbre de dossiers Linux

Les principaux répertoires

`/home`: répertoires personnels des utilisateurs

`/root`: répertoire personnel de l'utilisateur root

`/etc`: configuration du système

`/boot`: fichiers utiles au démarrage du système

`/bin` et `/sbin`: exécutables (binaires), programmes

`/lib`: librairies partagées (code réutilisable pour les programmes)

`/opt`: logiciels additionnels hors standard

`/media` et `/mnt`: points de montage pour les périphériques de stockage

`/tmp`: fichiers temporaires (supprimés régulièrement)

L'arbre de dossiers Linux

Les autres répertoires

- `/usr`: ressources partagées (quasiment tout ce qui est installé)
- `/dev`: périphériques (interfaces vers le matériel)
- `/lost+found`: données récupérées après une erreur de disque
- `/proc` et `/sys`: état du système et des processus en cours
- `/run`: informations sur les tâches en cours
- `/srv`: données pour les services fournis par le système
- `/var`: variables qui changent régulièrement (logs, données d'applicaions)

Exemple: `/proc/cpuinfo` contient des informations sur le processeur

Le répertoire home

Home sweet home

Retour au home, lister le répertoire avec **`ls`** `-la`

Principaux fichiers et dossiers cachés:

- `.profile`: script exécuté à la connexion d'une session
- `.bashrc`: script exécuté à l'ouverture d'un terminal
- `.bash_history`: historique du terminal
- `.config`: fichiers de configuration locaux
- `.cache`: cache et données des applications locales
- `.local`: équivalent de `/usr` pour les fichiers et programmes locaux

Tip: tout fichier ou dossier dont le nom commence par `.` est caché

Déplacer des fichiers

`mkdir` *`nom_dossier`* (make directory): créer un nouveau dossier

`mv` *`source destination`* (move): déplacer/renommer un fichier ou un dossier

`cp` *`source destination`* (copy): copier un fichier

`rm` *`fichier`* (remove): supprimer un fichier, *définitivement!*

Excercice:

1. déplacez-vous où vous avez crée votre fichier texte avec vos noms
2. créez un sous-dossier et copiez le fichier dans ce sous-dossier
3. vérifiez que la manipulation a marché avec l'interface graphique
4. supprimez ce dossier

Tip: **`cp`** et **`rm`** ont besoin de l'option `-r` (récursif) pour les dossiers

Afficher le contenu d'un fichier

`cat` *`fichier`* : afficher le contenu d'un fichier

`less` *`fichier`* : afficher le contenu d'un fichier page par page, avec navigation

- flèches pour se déplacer ligne par ligne
- `espace` pour avancer d'une page
- b pour reculer d'une page
- q pour quitter

`cat` est plus simple, mais **`less`** est plus pratique pour les fichiers longs

Tip: utiliser `Ctrl`+C pour interrompre une commande en cours (ex: **`cat`** sans argument)

Modifier un fichier

`nano` *`fichier`* : éditeur de texte en ligne de commande

- flèches pour naviguer
- `Ctrl+O` pour enregistrer
- `Ctrl+X` pour quitter

Tip: il existe d'autres éditeurs plus avancés (vim, emacs), mais nano est plus simple pour débuter et souvent disponible par défaut

Exercice de synthèse

Édition et affichage de fichiers texte

1. créez un fichier texte avec **`nano`**
2. collez 30 paragraphes de *Lorem ipsum* dans ce fichier https://loremipsum.io/generator/
3. affichez le contenu de ce fichier avec **`cat`**
4. affichez le contenu de ce fichier avec **`less`**

Tip: `Ctrl+C` et `Ctrl+V` pour copier/coller sont remplacés par `Ctrl+Shift+C` et `Ctrl+Shift+V` dans le terminal!

Des outils pour se simplifier la vie

Gérer les processus en cours

`htop`: gestionnaire des tâches dans le terminal (`q` pour quitter)

Affiche:

- utilisation du CPU et de la mémoire
- temps d'activité du système
- numéro du processus (PID) et utilisateur responsable du processus
- temps d'exécution
- commande associée au processus

Permet de tuer un processus avec `F9` (`SIGKILL`)

Exercice: ouvrez le navigateur web, vérifiez l'impact en ressources puis tuez le processus

Mettre à jour le système

`apt` (Advanced Package Tool): gestionnaire de paquets de Raspberry Pi OS

Un *paquet* peut contenir un programme, une librairie, des données

Utilisé avec une sous-commande:

- `update`: met à jour la liste des paquets disponibles
- `upgrade`: met à jour les paquets installés
- `install` *`paquet`*: installe un paquet
- `remove` *`paquet`*: désinstalle un paquet

Tip: certaines commandes comme **`apt`** nécessitent les droits d'administrateur

Les pleins pouvoirs

Un grand pouvoir implique de grandes responsabilités

`sudo` (superuser do): exécuter ce qui suit avec les droits d'administrateur

→ donne temporairement les mêmes permissions que *root*

Mettre à jour le système:

```
$ sudo apt update
$ sudo apt upgrade
```

Le danger de **`sudo`**

Avec les permissions de *root*, on peut causer des dégâts considérables

Par exemple, ne **JAMAIS** faire: **`sudo rm`** `-rf /`, cela supprimerait tout le système!

Un explorateur de fichiers en ligne de commande

`ranger` est un explorateur de fichiers visuel dans le terminal

- flèches pour naviguer
- `Enter` ou → pour ouvrir un fichier ou entrer dans un dossier
- `Shift+S` pour ouvrir un terminal dans le dossier courant
- `q` pour quitter
- `:help` pour ouvrir un menu d'aide

`ranger` n'est pas installé par défaut:

```
$ sudo apt install ranger
$ ranger
```

Exercice: retrouvez et visualisez votre fichier *Lorem ipsum* avec ranger

Un pense-bête pour les commandes

Ça commence à faire beaucoup à retenir

`man` *`commande`* (manual): afficher la documentation d'une commande

- rappel des options et arguments disponibles
- navigation identique à **`less`**
- avantage: super complet
- inconvénient: trop complet

Exemple: **`man ls`** $\rightarrow$ pas super clair pour un débutant

Un pense-bête encore plus bête

`cheat` *`commande`*: pense-bête simplifié pour les commandes

avec exemples d'utilisation courants

beaucoup plus court que **`man`**

Pas installé par défaut, et pas disponible via **`apt`**!

Installation avec **`snap`** (gestionnaire de paquets alternatif):

```
$ sudo apt install snapd
$ sudo snap install snapd
$ sudo snap install cheat
$ snap run cheat ls
```

C'est un peu long de taper **`snap run cheat`** à chaque fois non?

Où va-t-on chercher les commandes?

Les commandes sont cherchées dans les dossiers listés dans la variable `PATH`

Les variables

Le terminal utilise des variables pour stocker des informations et les réutiliser ailleurs

- `NOM="valeur"` pour créer une variable
- `$NOM` pour accéder à la valeur d'une variable

Attention pas d'espaces!

`echo` *`argument`*: afficher l'argument dans le terminal

Exercice: analyser la valeur de la variable `PATH` avec **`echo`** `$PATH`

Modifier le PATH par défaut

Objectif: pouvoir lancer **`cheat`** sans taper **`snap`** `run cheat`

Comment? Ajouter le dossier d'installation de **`snap`** (`/snap/bin`) au PATH

1. ouvrir le fichier `~/.profile`
2. ajouter la ligne `PATH="$PATH:/snap/bin"`
3. sauvegarder, puis fermer la session
4. ouvrir une nouvelle session et afficher la valeur de PATH
5. vérifier que `cheat` est maintenant utilisable directement

Pourquoi? À chaque connexion de l'utilisateur, l'exécution de `.profile` ajoute `/snap/bin/` à la fin du PATH existant

Contrôler le terminal à distance

SSH

Secure Shell: accéder à son terminal à distance

Depuis un terminal sur votre ordinateur: **`ssh`** *`utilisateur@adresse`*

- afficher son adresse IP: **`hostname`** `-I`
- utilisateur: `rpi`
- mot de passe: `rpi`

Fermeture d'un terminal: `exit` ou `Ctrl+D`

En fermant, toutes les commandes en cours dans ce terminal sont tuées

Vrai aussi pour les sessions SSH → attention aux commandes qui prennent du temps!

En cas de perte de connexion, la session SSH est fermée après un délai

Utilisation du nom d'hôte pour se connecter

Plutôt que d'utiliser l'IP qui peut changer, on peut utiliser son nom s'il est unique:

1. modifier `/boot/firmware/user-data` pour changer le nom de la machine
2. redémarrer le Raspberry Pi
3. se connecter avec le nouveau nom: **`ssh`** *`rpi@hostname.local`*

Pour aller plus loin, on peut configurer des clés SSH pour se connecter sans mot de passe

Tip: utiliser **`sudo reboot`** pour redémarrer depuis le terminal

Cockpit

Une solution web de contrôle à distance

Une interface web pour gérer votre serveur à distance:

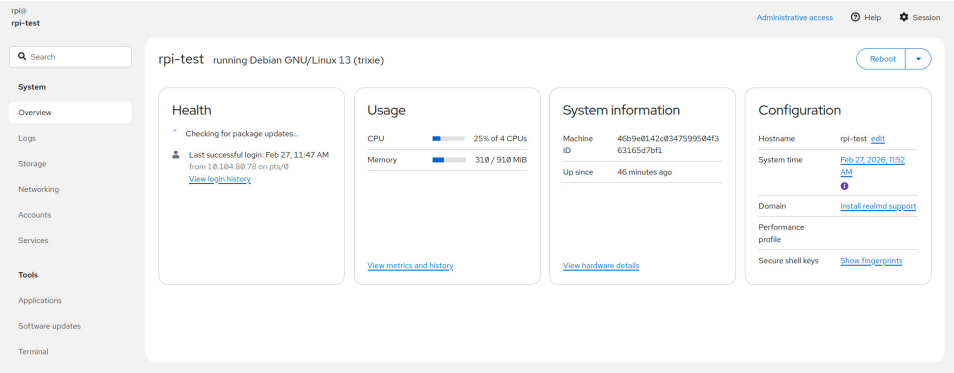
- simple à utiliser (visuel)
- accessible depuis n'importe quel navigateur

1. installez Cockpit: **`sudo apt`** `install cockpit`
2. connectez-vous à l'interface web dans un navigateur: `localhost:9090`
3. vérifiez l'état du système (CPU, mémoire, stockage) et comparez avec **`htop`**
4. essayez d'y accéder à distance avec l'adresse IP (toujours dans un navigateur)

Tip: Cockpit permet d'installer des "applications" pour ajouter des fonctionnalités (ex: installation de paquets, exploration de fichiers)

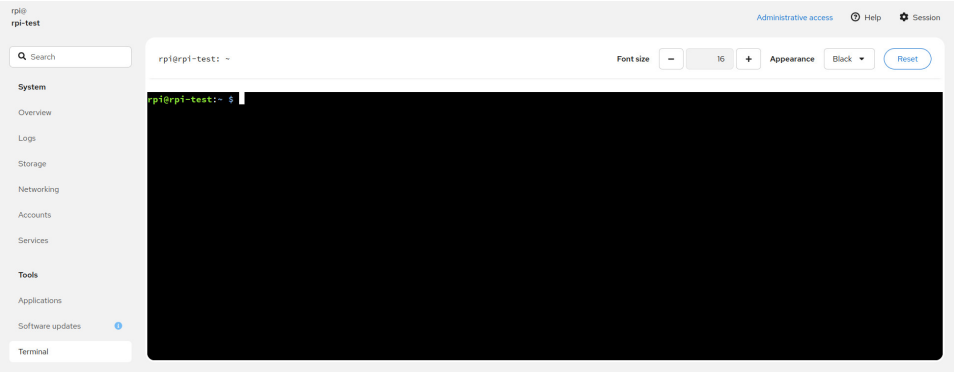
Cockpit

Vue d’ensemble sur le système



Cockpit

Terminal à distance intégré



Tmux

Garde une session active en arrière plan → parfait pour des mises à jour en SSH

`sudo apt` *`install tmux`*

`tmux`: lancer une session tmux

`Ctrl`+B puis D: détacher la session (elle reste active en arrière plan)

`tmux` *a* (attach): rejoindre la dernière session

Exercice:

1. ouvrez un document avec **`less`** dans le terminal cockpit
2. déconnectez internet un instant. **`less`** est-il toujours ouvert après reconnexion?
3. répetez l'exercice, mais en utilisant tmux pour préserver votre session
4. détachez la session pendant que **`less`** est ouvert, et rejoignez-la depuis un terminal SSH, puis depuis le terminal local

Tmux

Bonus: plusieurs terminaux en un

Tmux est aussi un multiplexeur → plusieurs terminaux dans une session!

- `Ctrl+B` puis `"` ou `%` pour diviser la fenêtre en panneaux
- `Ctrl+B` puis flèches pour naviguer entre les panneaux
- `Ctrl+B` puis `C` pour créer une nouvelle fenêtre
- `Ctrl+B` puis `W` pour naviguer entre les fenêtres et sessions
- **`tmux`** *`ls`* pour lister les sessions tmux en cours

Et encore bien d'autres fonctions (voir `Ctrl+B` puis `?`)